Functional Versus Imperative Is Solved

Power, Cost, and Shared State in the Paradigm Debate

29 October 2026

1 Two Ways to Sum a List

Here is the same function written twice.

Listing 1: Imperative, in C.

```c
int total(const int *xs, int n) {
  int acc = 0;
  for (int i = 0; i < n; i++)
    acc += xs[i];
  return acc;
}
```

Listing 2: Functional, in Haskell.

```haskell
total :: [Int] -> Int
total []       = 0
total (x : xs) = x + total xs
```

The first is a list of instructions. The second is a pair of equations. Programmers have argued about which is better for most of the history of programming, and the argument is older than most of the languages it is now conducted in. We settle it.

Definition 1.1 (Imperative)**.** A program is *imperative* if its meaning is a sequence of commands, each of which changes a *store*, a map from locations to values. Running it executes the commands in order, and its result is read from the final store.

Definition 1.2 (Functional)**.** A program is *functional* if its meaning is an expression, and the meaning of an expression is its value. It is *pure* if the value of every expression depends only on the values of its subexpressions, so that any subexpression can be replaced by its value without changing the meaning of the whole. Strachey, borrowing from Quine, called this property *referential transparency* [2].

Remark 1.3. The term has since been used in several senses that are not equivalent [1]. We use Strachey's. Both definitions classify programs, not languages. C permits a function that assigns nothing, and Haskell permits one that does nothing but assign. Most languages in use permit both, so the question is not which language to use but which kind of program to write.

References

[1] H. Søndergaard and P. Sestoft, Referential transparency, definiteness and unfoldability, *Acta Informatica* 27(6), 1990.

[2] C. Strachey, Fundamental concepts in programming languages, *Higher-Order and Symbolic Computation* 13(1–2), 2000. Lecture notes from 1967.